FAST – NUCES

Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 14 – Templates and Exception Handling

Objectives

- Implement template functions and classes
- Implement templates with multiple types
- Template classes and specialization
- Understand and apply C++ exception handling to manage runtime errors safely.
- Implement and use custom and built-in exceptions to handle invalid input, out-of-bounds access, and arithmetic errors.
- Develop programs that prevent crashes by properly throwing, catching, and responding to exceptions.

Tom & Jerry Templates

Welcome to the chaotic house of **Tom Cat** and **Jerry Mouse**!
Tom is always chasing, Jerry is always outsmarting — and you will use **C++ templates and exceptions** to help Jerry stay one step ahead.

Function Templates (Jerry's Smart Tricks)

Jerry uses clever tricks that work in every situation - whether comparing cheese sizes, counting snacks, or picking the best hiding spot.

A **template parameter** is like Jerry's secret trick that adapts depending on the situation.

Format:
template <class identifier> function_declaration;

Task 1

Jerry needs help comparing items before Tom grabs them!

a.

Write two function templates:

- **GetMax** - returns the larger of two values
- **GetMin** - returns the smaller of two values

These should work for any type of data (like cheese sizes, trap weights, or milk quantities).

b.

Paste the following code into your program and run it.
Everything should work smoothly - like Jerry escaping a trap.

```
int main ()
{
  int i=5, j=6, k;
  long l=10, m=5, n;
  k=GetMax<int>(i,j);
  n=GetMin<long>(l,m);
  cout << k << endl;
  cout << n << endl;
  return 0;
}
```

c.

Remove <int> and <long> from the function calls and run again.

Does the program still work, or does Tom finally catch Jerry?

d.

Replace your main with this one.

- Modify your GetMax and GetMin so this works correctly - like Jerry fixing a broken escape plan.

```
int main ()
{
  char i='Z';
  int  j=6, k;
  long l=10, m=5, n;
  k=GetMax<int,long>(i,m);
  n=GetMin<int,char>(j,l);
  cout << k << endl;
  cout << n << endl;
  return 0;
}
```

e.

Remove <int,long> and <int,char> from this version and re-run.
Does anything unexpected happen? Is Tom confused by mixed types?

Class Templates (Jerry's Storage Boxes)

Jerry collects cheese and stores it in different containers — but all containers work the same way.

Task 2

Create a **Pair class template** to store two values of the same type - like two pieces of cheese Jerry stole.

Specifications:

- Private data member: values (array of size 2, type T)
- Constructor that takes two parameters
- GetMax defined **inside** the class
- GetMin defined **outside** the class

Test with:

```
int main ()
{
  Pair <double> myobject (1.012, 1.01234);
  cout << myobject.getmax();
  return 0;
}
```

Template Specialization (Tom vs Jerry Behavior)

Sometimes things behave differently — just like Tom and Jerry react differently in the same situation.

Task 3

a.

Complete the **Container template**:

- General template:
    - Implement increase() (for numeric values)
- Specialized template for **char**:
    - Implement uppercase()
    - Do NOT use toupper()
    - Do NOT use inline functions

b.

Test using:

```
int main ()
{
  Container<int> myint (7);
  Container<char> mychar ('j');

  cout << myint.increase() << endl;
  cout << mychar.uppercase() << endl;

  return 0;
}
```

Non-Type Template Parameters (Jerry's Cheese Line)

Jerry lines up cheese pieces in a fixed row — the number of slots is known beforehand.

Task 4

a.

Implement the **Sequence class** that stores **N elements**.

- Use a **non-type template parameter** for size
- Do NOT use inline definitions

b.

Test using:

```
int main ()
{
  Sequence <int,5> myints;
  Sequence <double,5> myfloats;

  myints.setmember (0,100);
  myfloats.setmember (3,3.1416);

  cout << myints.getmember(0) << '\n';
  cout << myfloats.getmember(3) << '\n';

  return 0;
}
```

Task 5 - Storing Pair Inside Sequence

Now Jerry wants to store **pairs of cheese** inside his storage system — but Tom keeps interfering!

To make this work:

- Add a **default constructor** to your Pair class
- Overload the **<< operator** using a friend function

Test with:

```
int main ()
{
  Pair <double> y (2.23,3.45);
  Sequence <Pair<double>,5> myPairs;

  myPairs.setmember (0,y);

  cout << myPairs.getmember(0) << '\n';
}
```

Exception Handling (Tom's Chaos & Jerry's Safety)

Things often go wrong in the house — plates break, traps snap, and chaos happens!

Task 6 - Unexpected Trap

Tom triggers a trap that throws unexpected values!

- Write a program that:
    - Throws and catches an **integer exception**
    - Prints the value when caught
- Extend the program:
    - Add a catch block for **double**
    - Add a catch(...) for unknown chaos
- Let the user choose what type of value to throw (using if or switch)

Task 7 - Kitchen Mixing Disaster

Jerry is mixing ingredients, but if something invalid is added — disaster!

- Create a function:
  add(double a, double b)
- Requirements:
    - If input is non-numeric → throw std::invalid_argument
    - Use cin.fail() to detect invalid input
    - Catch the exception in main()
    - Display an error message

Task 8 – Out-of-Bounds Trouble

Tom tries to grab cheese from outside the shelf!

- Create:
  - Array of size 5
  - Function getElementAtIndex(int index)
- If index is invalid:
  - Throw custom exception: ArrayIndexOutOfBoundsException
- Requirements:
  - Inherit from std::exception
  - Add printMessage() method with a fun error message
  - Handle exceptions in main()

Task 9 – The Broken Recipe File

Jerry finds a secret recipe file, but Tom might have damaged it!

- Open a file (e.g., "tom_jerry_scroll.txt")
- Throw std::runtime_error if:
  - File cannot be opened
  - File is empty
- If file is valid:
  - Read and display contents
- Handle all exceptions in main()

Task 10 – Custom Exception (Ultimate Chaos Control)

Jerry is testing trap power levels - too low or too high causes problems!

- Create custom exception class:
  class TrapPowerException : public std::exception

Requirements:

- Constructor takes a string message
- Store the message
- Override:

  const char* what() const noexcept

In main():

- Ask user for a value (0–100)
- If out of range:
  - Throw exception with custom message
- Catch and display using e.what()

Hints (VERY important - read before coding)

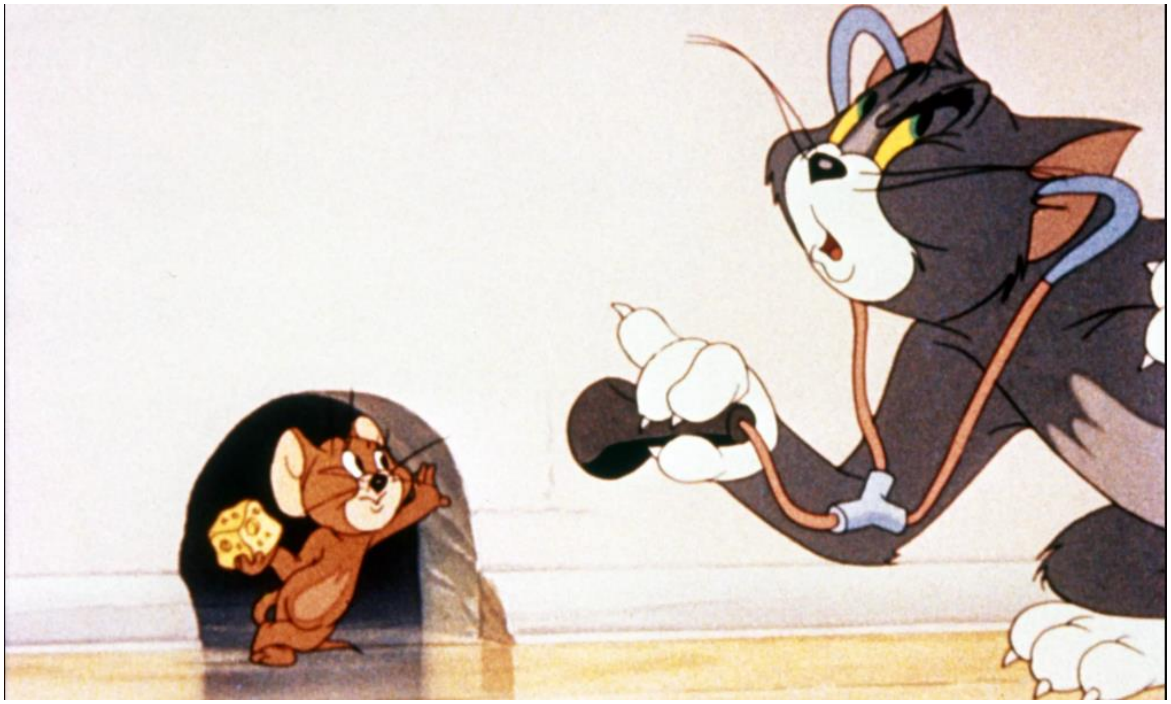
- The what() function MUST use this exact return type and signature when inheriting from std::exception:
  const char* what() const noexcept

Why?

- std::exception requires it
- what() must return a **const char***
- noexcept is required for exception safety

To return your stored message (which will be a std::string), use:

return message.c_str();



----------**Submission Instructions**----------

- Submit your tasks on Google Classroom within given deadline
- Also submit screenshots of output of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
- “.cpp” is the actual file where all your code is saved.
- Always submit .cpp files when asked for submission